

МИНОБРНАУКИ РОССИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА
ВЕЛИКОГО»**

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление 02.03.01 Математика и компьютерные науки

Отчёт по дисциплине «Математическая логика и теория автоматов»

Лабораторная работа №2 «Реализация
конечного автомата для проверки текста по
заданному регулярному выражению и функции
случайной генерации строки»

Вариант №18

Студент: _____

Черданцев Артём Сергеевич

Преподаватель: _____

Востров Алексей Владимирович

«____» _____ 20__ г.

Содержание

Введение	3
1 Математическое описание	4
1.1 Регулярные множества и регулярные выражения	4
1.2 Регулярное выражение заголовка e-mail	5
1.3 Языки и грамматики	5
1.4 Недетерминированный конечный автомат	6
1.5 Конечный автомат-распознаватель	6
1.6 Детерминированный конечный автомат	7
2 Особенности реализации	10
2.1 enum State	10
2.2 Функция validateEmail	10
2.3 Функция generateValidEmail	11
3 Результаты работы программы	13
Заключение	15
Список источников	16

Введение

Цель данной лабораторной работы заключалась в том, чтобы по заданному регулярному выражению построить вначале недетерминированный конечный автомат, затем детерминированный конечный автомат. Реализовать программу, которая проверяет введенный текст через реализацию конечного детерминированного автомата, а также реализовать функцию случайной генерации корректной строки по полученному конечному детерминированному автомату.

В данном отчете представлены детерминированный и недетерминированный конечные автоматы, построенные по заданному регулярному выражению «Разбор заголовка в e-mail». Также представлена программная реализация детерминированного конечного автомата на языке программирования Java. Программа работает как автомат-распознаватель, проверяя цепочки на принадлежность языку, и как автомат-генератор, порождающий цепочки заданного языка.

Регулярное выражение:

18. Разбор заголовка в e-mail

```
/\b[A-Z0-9._%+-]+@[?:[A-Z0-9-]+\.)+[A-Z]{2,6}\b/i
```

Рис. 1: Заданное регулярное выражение

В процессе реализации регулярное выражение было изменено следующим образом: $A-Z \rightarrow a-z$, так как e-mail адреса не могут содержать буквы верхнего регистра.

1 Математическое описание

1.1 Регулярные множества и регулярные выражения

Регулярные множества - это (бесконечные) множества цепочек, построенных из символов конечного словаря по определенным правилам.

Регулярные множества, как множества цепочек, построенные над конечным словарем (по определенным правилам) – это языки. Их называют регулярными языками.

Формально класс регулярных множеств над конечным словарем Σ , определяется так:

$\emptyset = \{\}$ - регулярное множество;

$\{\varepsilon\}$ - регулярное множество;

$\{a\}$ - регулярное множество для любого $a \in \Sigma$;

Если P и Q - регулярные множества, то регулярны следующие операции над ними:

1. Объединение $P \cup Q$
2. Произведение $P \times Q$
3. Итерация P^*

Регулярные выражения — формальный язык шаблонов для поиска и выполнения манипуляций с подстроками в тексте. Регулярное выражение показывает, как можно построить регулярное множество цепочек из одноэлементных множеств с использованием трех операций: конкатенации, объединения и итерации.

Регулярные множества определяют семантику регулярных выражений. Регулярные выражения представляют конечный порождающий механизм для регулярного языка.

Формально класс регулярных множеств над конечным словарем Σ , определяется так: $\emptyset$ - регулярное множество ε - и любой символ $a \in \Sigma$ - регулярные выражения;

Если R_1, R_2 - регулярные выражения, то регулярными выражениями будут:

1. их сумма $R_1 + R_2$
2. их произведение (конкатенация) $R_1 R_2$
3. итерация R^* и усеченная итерация R_1^+

По заданному регулярному выражению можно построить конечный автомат, так как классы регулярных множеств и автоматных языков совпадают (по теореме Клини).

Теорема Клини

Классы регулярных множеств и автоматных языков совпадают.

То есть любой язык, распознаваемый конечным автоматом, может быть задан формулой (регулярным выражением), и для любого языка, заданного регулярным выражением, может быть построен конечный автомат, его распознающий.

1.2 Регулярное выражение заголовка e-mail

$$\wedge b[a-z0-9._\%+-]+\@(\?:[a-z0-9-]+\backslash.)+[a-z]\{2,6\}\wedge b/i$$

Рис. 2: Заданное регулярное выражение

- $\wedge b$ - Начало шаблона слова.
- $[a-z0-9._\%+-]+\@$ - Локальная часть адреса электронной почты (часть до символа @).
 - $[a-z0-9._\%+-]$ - Разрешенные символы для локальной части адреса, которые могут быть буквами от а до z, цифрами от 0 до 9, а также некоторыми специальными символами ($._\%+-$).
 - '+' - Означает, что один или более разрешенных символов должны присутствовать.
- @ - Символ @, разделяющий локальную часть адреса электронной почты от доменной части.
- $(\?:[A-Z0-9-]+\backslash.)+$ - Этот блок определяет доменную часть адреса электронной почты.
 - $(\?: \dots)$ - Это немаркируемая группа для группировки паттерна.
 - $[A-Z0-9-]+\@$ - Разрешенные символы для имени домена, аналогичные локальной части, с добавлением дефисов (-).
 - $\backslash.$ - Литерал точки, разделяющий части домена.
 - + - Означает, что должно быть одно или более повторений такой группы символов с точкой.
- $[a-z]\{2,6\}$ - Этот блок определяет верхний уровень домена (TLD) в адресе электронной почты.
 - $[a-z]$ - Разрешенные символы для TLD.
 - $\{2,6\}$ - Определяет длину TLD от 2 до 6 символов.
- $\wedge b$ - Завершение шаблона с границей слова.

1.3 Языки и грамматики

Язык над конечным словарем V - это произвольное множество конечных цепочек над этим словарем - L_V .

Совокупность грамматических правил некоторого языка - **грамматика** - представляет из себя теоретическую модель языка, состоящую из отдельных правил (гипотез) структурной организации языка.

Порождающая грамматика языка L - это конечный набор правил, позволяющих строить все предложения, принадлежащие языку L , и применение которых не даёт ни одного предложения, не принадлежащего L .

С помощью распознающей грамматики задаётся критерий принадлежности произвольной цепочки данному языку. Фактически, **распознающая грамматика** - это некоторый алгоритм,

принимающий произвольную цепочку символов над словарём V и дающий на выходе один из двух возможных ответов: «данная грамматика принадлежит языку L » либо «данная цепочка не принадлежит языку L ».

1.4 Недетерминированный конечный автомат

Недетерминированный конечный автомат-распознаватель — пятерка объектов:

$A = \langle S, X, s_0, \sigma, F \rangle$, где:

- $S \neq \emptyset$ - конечное множество состояний;
- $X \neq \emptyset$ - конечное множество входных сигналов (входной алфавит);
- $s_0 \in S$ - множество начальных состояний;
- $\sigma : S \times X \rightarrow S$ - функция переходов;
- $F \subset S$ - множество финальных состояний.

Для заданного регулярного выражения был построен недетерминированный конечный автомат. На Рис. 3 представлена таблица переходов полученного недетерминированного конечного автомата.

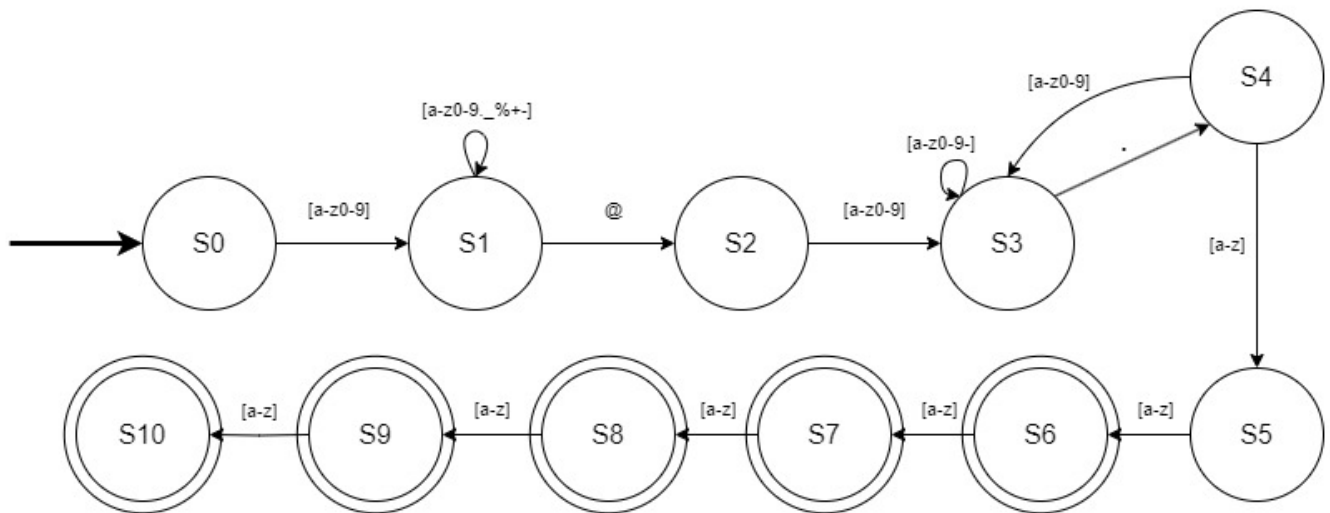


Рис. 3: Недетерминированный конечный автомат для заданного регулярного выражения

1.5 Конечный автомат-распознаватель

Недетерминированный конечный автомат-распознаватель — пятерка объектов:

$A = \langle S, X, s_0, \sigma : S \times X \rightarrow S, F \rangle$, где:

- $S \neq \emptyset$ - конечное множество состояний;
- $X \neq \emptyset$ - конечное множество входных сигналов (входной алфавит);
- $s_0 \in S$ - множество начальных состояний;
- $\sigma : S \times X \rightarrow S$ - функция переходов;
- $F \subset S$ - множество финальных состояний.

Конечный автомат распознаватель $A = \langle S, X, s_0, \sigma, F \rangle$, допускает входную цепочку $\alpha \in X^*$, если α переводит его из начального состояния в одно из конечных. Множество всех цепочек, допускаемых автоматом A , образует язык, допускаемый A .

1.6 Детерминированный конечный автомат

Детерминированный конечный автомат – автомат, в котором для каждого состояния определен переход по каждому из входных сигналов и при том только один, отсутствуют ε - переходы.

Любой недетерминированный конечный автомат-распознаватель можно детерминировать, то есть построить эквивалентный детерминированный конечный автомат-распознаватель.

В данном случае детерминирование автомата осуществлялось с помощью добавления переходов в состояние ошибки: если не указан переход из состояния по символу из алфавита, значит переход осуществляется в состояние ошибки.

Недетерминированный конечный автомат был приведен к конечному автомату без неоднозначных переходов. На Рис. 4 представлен результат первого шага детерминирования.

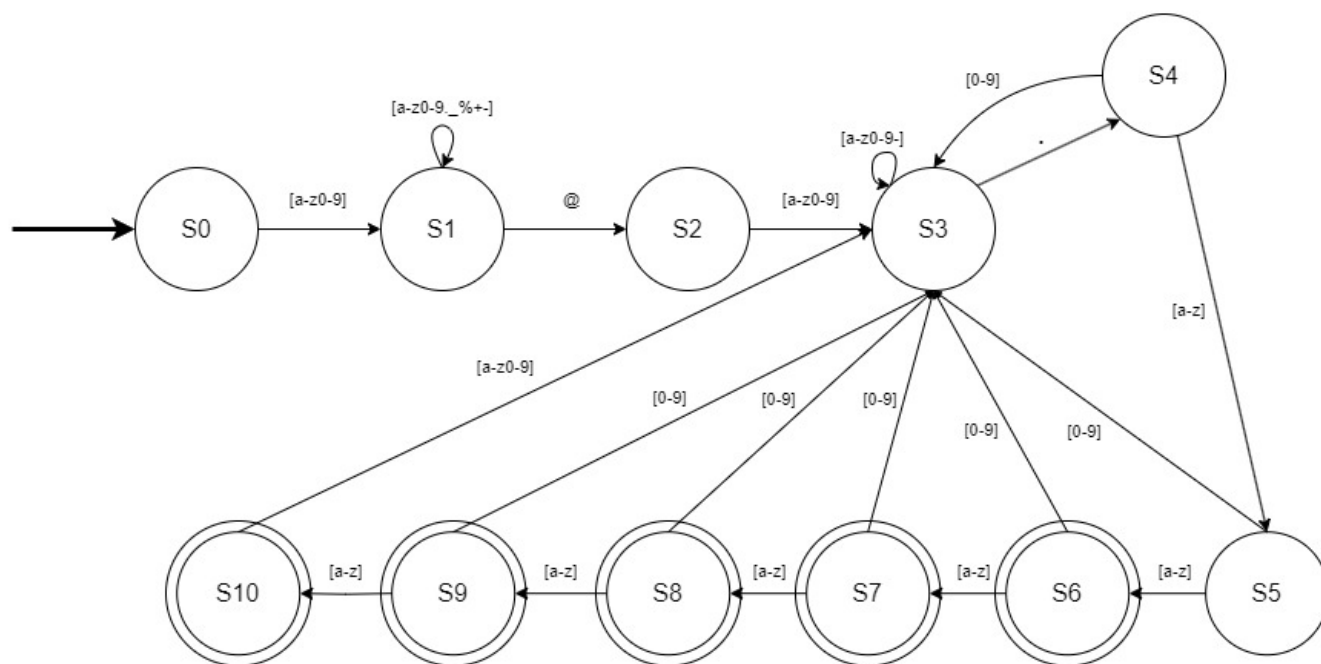


Рис. 4: Недетерминированный конечный автомат без неоднозначных переходов

На Рис. 5 представлен Детерминированный конечный автомат.

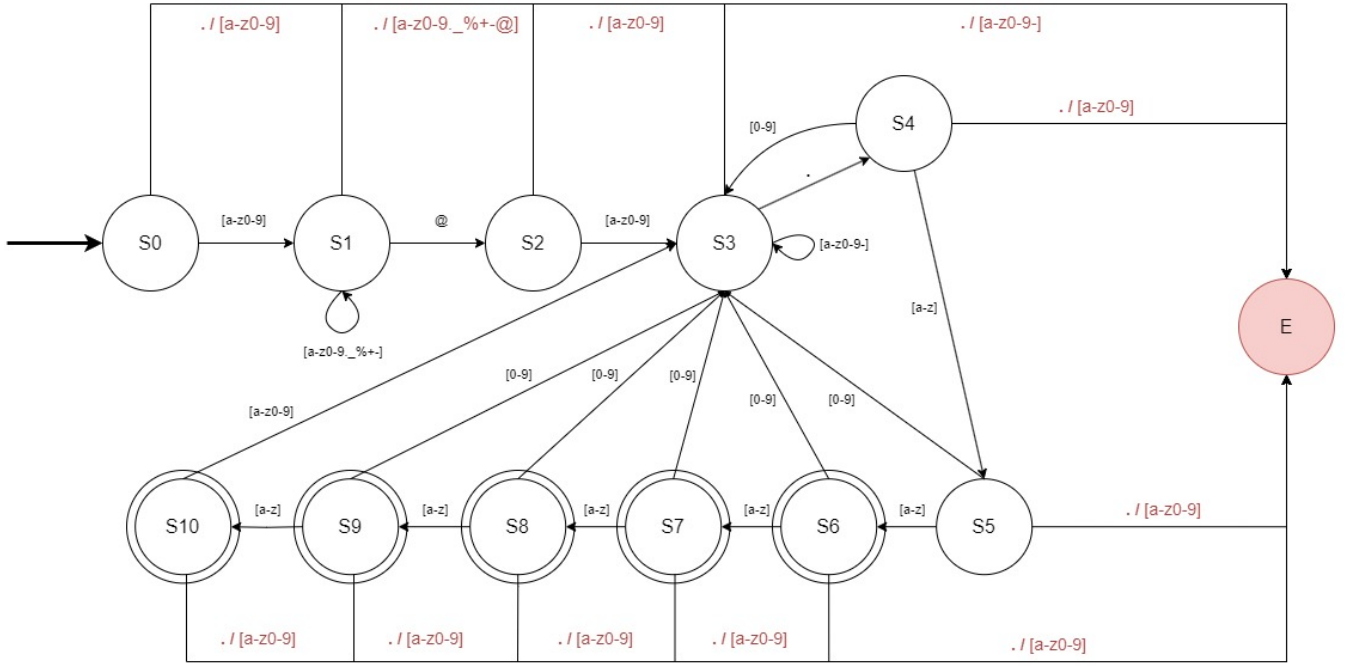


Рис. 5: Детерминированный конечный автомат

Формально автомат задаётся как:

- $S = S_0, S_1, S_2, S_3, S_4, S_5, S_6, S_7, S_8, S_9, S_{10}, S_E$
- $s_0 = S_0$
- $X = \{\backslash w, @, _, \%, +, -, \cdot\}$

Причём $\backslash w$ - любые буквы и цифры.

- $\sigma = \{$

Из вершины S_0 :

$$\begin{array}{llll} \sigma(S_0, \backslash w) = S_1, & \sigma(S_0, _) = S_E, & \sigma(S_0, +) = S_E, & \sigma(S_0, \cdot) = S_E, \\ \sigma(S_0, @) = S_E, & \sigma(S_0, \%) = S_E, & \sigma(S_0, -) = S_E, & \end{array}$$

Из вершины S_1 :

$$\begin{array}{llll} \sigma(S_1, \backslash w) = S_1, & \sigma(S_1, _) = S_1, & \sigma(S_1, +) = S_1, & \sigma(S_1, \cdot) = S_1, \\ \sigma(S_1, @) = S_2, & \sigma(S_1, \%) = S_1, & \sigma(S_1, -) = S_1, & \end{array}$$

Из вершины S_2 :

$$\begin{array}{llll} \sigma(S_2, \backslash w) = S_3, & \sigma(S_2, _) = S_E, & \sigma(S_2, +) = S_E, & \sigma(S_2, \cdot) = S_E, \\ \sigma(S_2, @) = S_E, & \sigma(S_2, \%) = S_E, & \sigma(S_2, -) = S_E, & \end{array}$$

Из вершины S_3 :

$$\begin{array}{llll} \sigma(S_3, \backslash w) = S_3, & \sigma(S_3, _) = S_E, & \sigma(S_3, +) = S_E, & \sigma(S_3, \cdot) = S_4, \\ \sigma(S_3, @) = S_E, & \sigma(S_3, \%) = S_E, & \sigma(S_3, -) = S_3, & \end{array}$$

Из вершины S_4 :

$$\begin{array}{llll} \sigma(S_4, \backslash w) = S_3, S_5, & \sigma(S_4, _) = S_E, & \sigma(S_4, +) = S_E, & \sigma(S_4, \cdot) = S_E, \\ \sigma(S_4, @) = S_E, & \sigma(S_4, \%) = S_E, & \sigma(S_4, -) = S_E, & \end{array}$$

Из вершины S_5 :

$$\begin{array}{llll} \sigma(S_5, \backslash w) = S_3, S_6, & \sigma(S_5, _) = S_E, & \sigma(S_5, +) = S_E, & \sigma(S_5, \cdot) = S_E, \\ \sigma(S_5, @) = S_E, & \sigma(S_5, \%) = S_E, & \sigma(S_5, -) = S_E, & \end{array}$$

Из вершины S_6 :

$$\begin{array}{llll} \sigma(S_6, \backslash w) = S_3, S_7, & \sigma(S_6, _) = S_E, & \sigma(S_6, +) = S_E, & \sigma(S_6, \cdot) = S_E, \\ \sigma(S_6, @) = S_E, & \sigma(S_6, \%) = S_E, & \sigma(S_6, -) = S_E, & \end{array}$$

Из вершины S_7 :

$$\begin{array}{llll} \sigma(S_7, \backslash w) = S_3, S_8, & \sigma(S_7, _) = S_E, & \sigma(S_7, +) = S_E, & \sigma(S_7, \cdot) = S_E, \\ \sigma(S_7, @) = S_E, & \sigma(S_7, \%) = S_E, & \sigma(S_7, -) = S_E, & \end{array}$$

Из вершины S_8 :

$$\begin{array}{llll} \sigma(S_8, \backslash w) = S_3, S_9, & \sigma(S_8, _) = S_E, & \sigma(S_8, +) = S_E, & \sigma(S_8, \cdot) = S_E, \\ \sigma(S_8, @) = S_E, & \sigma(S_8, \%) = S_E, & \sigma(S_8, -) = S_E, & \end{array}$$

Из вершины S_9 :

$$\begin{array}{llll} \sigma(S_9, \backslash w) = S_3, S_{10}, & \sigma(S_9, _) = S_E, & \sigma(S_9, +) = S_E, & \sigma(S_9, \cdot) = S_E, \\ \sigma(S_9, @) = S_E, & \sigma(S_9, \%) = S_E, & \sigma(S_9, -) = S_E, & \end{array}$$

Из вершины S_{10} :

$$\begin{array}{llll} \sigma(S_{10}, \backslash w) = S_3, & \sigma(S_{10}, _) = S_E, & \sigma(S_{10}, +) = S_E, & \sigma(S_{10}, \cdot) = S_E, \\ \sigma(S_{10}, @) = S_E, & \sigma(S_{10}, \%) = S_E, & \sigma(S_{10}, -) = S_E, & \end{array}$$

}

- $F = \{S_6, S_7, S_8, S_9, S_{10}\}$

2 Особенности реализации

2.1 enum State

State представляет различные состояния, через которые проходит конечный автомат при обработке текста (адреса электронной почты). Каждое состояние определяет, что программа ожидает увидеть на данном этапе обработки текста (например, символы имени пользователя, символ "@" или символы домена).

```
1 private enum State {
2     START,
3     USERNAME,
4     AT_SYMBOL,
5     DOMAIN,
6     DOT,
7     TOP_LEVEL_DOMAIN,
8     INVALID
9 }
```

2.2 Функция validateEmail

Функция **validateEmail** представляет собой реализацию детерминированного конечного автомата. Он проверяет введённую строку, используя методы: *isValidUsernameCharacter(char)*, *isValidDomainCharacter(char)*, *isValidTopLevelDomainCharacter(char)* и хранит текущее состояние автомата. Он итеративно проходит по каждому символу входного текста и переходит между различными состояниями State.

Вход: Введённая строка.

Выход: Сообщение о соответствии или несоответствии строки регулярному выражению.

```
1 public static void validateEmail(String text) {
2     State currentState = State.START;
3
4     for (char ch : text.toCharArray()) {
5         switch (currentState) {
6             case START:
7                 if (isValidUsernameCharacter(ch)) {
8                     currentState = State.USERNAME;
9                 } else {
10                    currentState = State.INVALID;
11                }
12                break;
13             case USERNAME:
14                 if (isValidUsernameCharacter(ch)) {
15                     currentState = State.USERNAME;
16                 } else if (ch == '@') {
17                     currentState = State.AT_SYMBOL;
18                 } else {
19                     currentState = State.INVALID;
20                 }
21             }
22         }
23     }
```

```

21         break;
22     case AT_SYMBOL:
23         if (isValidDomainCharacter(ch)) {
24             currentState = State.DOMAIN;
25         } else {
26             currentState = State.INVALID;
27         }
28         break;
29     case DOMAIN:
30         if (isValidDomainCharacter(ch)) {
31             currentState = State.DOMAIN;
32         } else if (ch == '.') {
33             currentState = State.DOT;
34         } else {
35             currentState = State.INVALID;
36         }
37         break;
38     case DOT:
39         if (isValidTopLevelDomainCharacter(ch)) {
40             currentState = State.TOP_LEVEL_DOMAIN;
41         } else {
42             currentState = State.INVALID;
43         }
44         break;
45     case TOP_LEVEL_DOMAIN:
46         if (isValidTopLevelDomainCharacter(ch)) {
47             currentState = State.TOP_LEVEL_DOMAIN;
48         } else {
49             currentState = State.INVALID;
50         }
51         break;
52     case INVALID:
53         break;
54     }
55 }
56
57 // Проверка окончательного состояния
58 if (currentState == State.TOP_LEVEL_DOMAIN) {
59     System.out.println("Строка соответствует");
60 } else {
61     System.out.println("Строка не соответствует");
62 }
63 }

```

2.3 Функция generateValidEmail

Функция `generateValidEmail` генерирует случайный верный адрес электронной почты с использованием конечного автомата. Он начинается с генерации случайного имени пользователя, затем добавляет символ "@" и генерирует доменную часть, включая поддомены и домен верхнего уровня, чтобы получить полный адрес электронной почты.

Вход: -

Выход: Сгенерированная строка, соответствующая заданному регулярному выражению.

```
1 private static String generateValidEmail() {
2     Random random = new Random();
3     StringBuilder stringBuilder = new StringBuilder();
4
5     int length = random.nextInt(10) + 5;
6     for (int i = 0; i < length; i++) {
7         char character;
8         if (i == 0) {
9             character = (char) ('a' + random.nextInt(26)); // Первый символ - буква
10        } else {
11            character = getRandomUsernameCharacter(random); // Остальные символы - буквы
12        }
13        stringBuilder.append(character);
14    }
15    stringBuilder.append("@");
16
17    length = random.nextInt(7) + 3; // Длина домена от 3 до 9 символов
18    for (int i = 0; i < length; i++) {
19        char character = (char) ('a' + random.nextInt(26)); // Буква
20        stringBuilder.append(character);
21    }
22    stringBuilder.append('.');
23
24    length = random.nextInt(5) + 2; // Длина TLD от 2 до 6 символов
25    for (int i = 0; i < length; i++) {
26        char character = (char) ('a' + random.nextInt(26)); // Буква
27        stringBuilder.append(character);
28    }
29
30    return stringBuilder.toString();
31 }
```

3 Результаты работы программы

На рис. 6 представлена работа программы при проверке введенной строки.

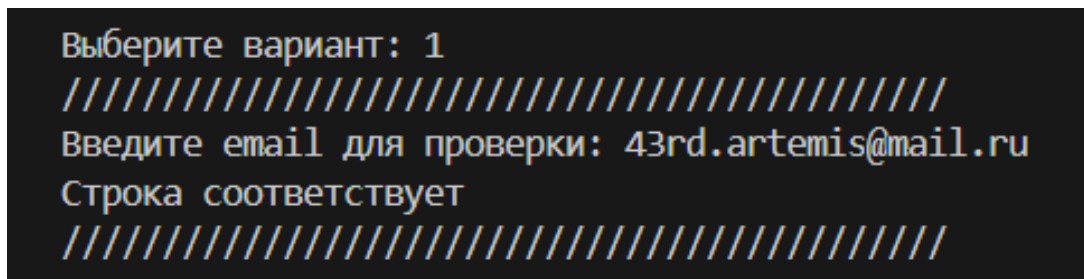


Рис. 6: Проверка введенной строки

На рис. 7 представлена работа программы при проверке некорректной введенной строки.

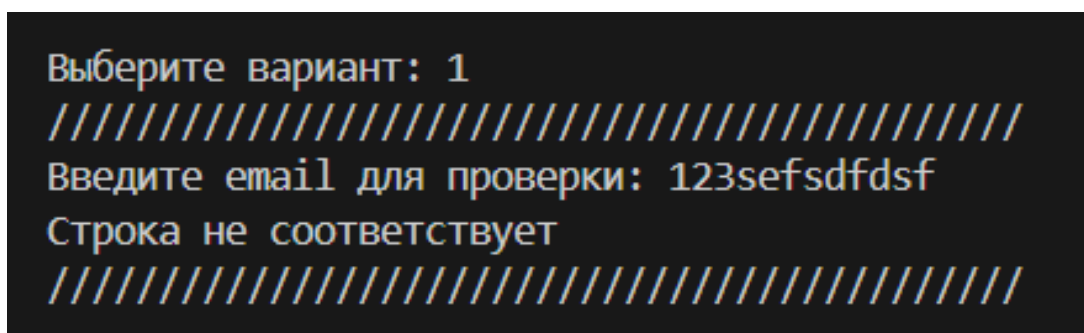


Рис. 7: Проверка некорректной введенной строки

На рис. 8 представлена работа программы при вводе пустой строки.

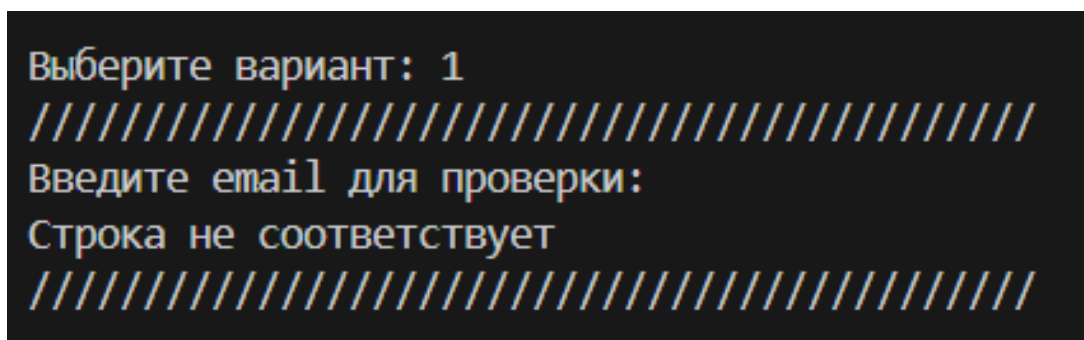


Рис. 8: Проверка пустой строки

На рис. 9 представлена работа программы при генерации e-mail адреса.

```
Выберите вариант: 2
////////////////////////////////////////
Сгенерированный email: c.b.mq@unvqls.darxtl
////////////////////////////////////////

Выберите вариант: 2
////////////////////////////////////////
Сгенерированный email: oba+_cieb8217p@yrnpsw.vlwb
////////////////////////////////////////
```

Рис. 9: Генерация e-mail адреса

На рис. 10 представлена работа программы при вводе строки, содержащей символы не из алфавита.

```
Выберите вариант: 1
////////////////////////////////////////
Введите email для проверки: фывфыв
Строка не соответствует - Строка содержит символы не из алфавита
////////////////////////////////////////
```

Рис. 10: Обработка символов не из алфавита

Заключение

В результате выполнения данной лабораторной работы был реализован конечный автомат по заданному регулярному выражению. Для заданного регулярного выражения, описывающего регулярный язык, был построен недетерминированный конечный автомат, который затем был преобразован в детерминированный.

Далее на основе детерминированного автомата была разработана программа на языке программирования Java. Эта программа функционирует как распознаватель-автомат, проверяя цепочки на принадлежность к языку, а также как автомат-генератор, порождающий цепочки данного языка.

Преимущества:

- Реализованы автоматические тесты проверяющие функцию проверки введенной строки;
- Программа может работать с детерминированными конечными автоматами с различными алфавитами и количеством состояний, в коде предусмотрена возможность добавления новых состояний и символов для проверки введенной строки и генерации.

Недостатки:

- Сгенерированные e-mail адреса синтаксически правильные, но соответствуют реальным доменам;
- В программе нет обработки доменов имеющих количество уровней больше, чем 2.

Масштабируемость:

- Расширить конечный автомат, чтобы контролировать многоуровневые домены.

Список источников

1. Теория автоматов Ю.Г. Карпов - СПб.: Питер, 2003. - 208 с.
2. Теория и технология программирования. Основы построения трансляторов. Карпов Ю. Г. — СПб.: БХВ-Петербург, 2005. — 272 с.: ил.